Smart Contract Security Assessment

Launchly | Lotique Labs

Date of Engagement: 02.20.2025 — 02.25.2025

Auditors: Lotique Labs Security Team

ALL FINDINGS	CRITICAL	HIGH
25	3	6
MEDIUM	**LOW**	**INFORMATIONAL**
4	11	1

Table of Contents

1. Introduction 3
2. Assessment Summary 4
3. Test Approach and Methodology 6
4. Risk Methodology 7
5. Scope 9
6. Assessment Summary & Findings Overview 10
7. Findings & Technical Details 12

1. Introduction

```
Launchly engaged Lotique Labs to conduct a security assessment on their smart
contracts beginning on February 20th, 2025 and ending on February 25th, 2025.
The security assessment was scoped to the smart contracts provided to Lotique
Labs. Commit hashes and additional details are available in the Scope section
of this report.
```

Launchly is a token factory and bonding curve protocol deployed on the Binance Smart Chain (BSC). The platform enables users to create ERC-20 meme tokens with configurable metadata and anti-sniper protections, paired with linear bonding curves that facilitate decentralized price discovery using BNB as the quote asset. The system integrates Chainlink price feeds for BNB/USD conversion and supports both manual and atomic token-curve deployment flows.

2. Assessment Summary

2.1 Resource Allocation

Lotique Labs was provided 5 days for the engagement and assigned 1 full-time security engineer to review the security of the smart contracts in scope. The engineer is a blockchain and smart contract security expert with advanced skills in penetration testing and smart contract exploitation.

2.2 Objectives

The objectives of the assessment were to:

Identify potential security issues within the smart contracts

Ensure that smart contract functionality operates as intended

Validate the mathematical correctness of the bonding curve pricing model

Assess the safety of token factory deployment and initialization flows

2.3 Executive Summary

Lotique Labs identified several critical and high-severity areas for improvement to reduce the likelihood and impact of security risks:

```
Fix the bonding curve pricing formula to correctly compute the quadratic term
without the extraneous /WAD division that nullifies price progression
```

Refactor the token factory to mint tokens to the factory contract and distribute atomically, preventing revert conditions and ensuring proper curve funding

```
Scale curveAllocation by token decimals to prevent orders-of-magnitude
pricing errors
```

```
Implement slippage protection in the sell() function by uncommenting and
enforcing the minQuoteOut parameter
```

```
Add Chainlink oracle staleness validation using updatedAt timestamps with
appropriate threshold checks
```

Exclude bonding curve contracts from MemeToken anti-sniper limits to prevent global rate-limiting of curve operations

Restrict owner rescue functions to prevent potential rug-pull scenarios

Enforce 18-decimal token requirements for bonding curve integrations to prevent mathematical breakage

```
Refactor internal function calls to preserve msg.sender context across
factory convenience methods
```

Remove unused state variables, events, and constants to reduce contract surface area and gas costs

```
Validate that feeRecipient can receive BNB before setting it as an immutable
parameter to prevent permanent denial of service
```

```
Add deadline parameters to both buyWithBNB() and sell() to prevent stale
transactions from executing at outdated prices
```

Adhere to the checks-effects-interactions pattern by updating state variables before performing external calls

3. Test Approach and Methodology

Lotique Labs performed a combination of manual and automated security testing to balance efficiency, timeliness, practicality, and accuracy. The following phases were undertaken:

Research and Architecture Review: In-depth research into the architecture, purpose, and use of the Launchly platform, including the bonding curve mathematical model, token factory patterns, and Chainlink oracle integration

Manual Code Review: Smart contract manual code review and line-by-line walkthrough of all five contracts in scope

Logic and Connectivity Analysis: Graphing out functionality and contract logic, connectivity, and function call flows between the factory, token, and bonding curve contracts

Arithmetic Vulnerability Assessment: Manual assessment of use and safety for critical Solidity variables and functions to identify arithmetic-related vulnerability classes, with particular focus on WAD-scaled fixed-point arithmetic in the bonding curve

Local Testing: Local testing and simulation of buy/sell operations using custom Hardhat scripts

Static Security Analysis: Static security analysis of all contracts in scope

4. Risk Methodology

Lotique Labs uses an adapted Blockchain Vulnerability Scoring System (BVSS) to assess the severity of identified findings. The BVSS considers three categories of metrics to compute a final severity score.

4.1 Exploitability Metrics

The exploitability score is determined by the product of the following metrics:

Metric	Symbol	Values
Attack Origin	AO	A (Arbitrary — any account) / S (Specific — limited access)
Attack Cost	AC	L (Low) / M (Medium) / H (High)
Attack Complexity	AX	L (Low) / M (Medium) / H (High)

```
Formula: E = AO × AC × AX
```

4.2 Impact Metrics

The impact score considers the maximum individual impact and an adjusted average of remaining impacts:

Metric	Symbol	Values
Confidentiality	C	N / L / M / H / C
Integrity	I	N / L / M / H / C
Availability	A	N / L / M / H / C
Deposit	D	N / L / M / H / C
Yield	Y	N / L / M / H / C

```
Formula: I = max(mI) + Σ(mI - max(mI)) / 4
```

4.3 Severity Coefficients

Metric	Symbol	Values
Reversibility	R	N (None — irreversible) / P (Partial) / F (Full)
Scope	S	U (Unchanged — isolated) / C (Changed — extends beyond component)

```
Formula: C = R × S
```

4.4 Final Score

```
Severity = min(10, E × I × C × 10)
```

Severity	Score Range
Critical	9.0 — 10.0
High	7.0 — 8.9
Medium	4.5 — 6.9
Low	2.0 — 4.4
Informational	0.0 — 1.9

5. Scope

5.1 Repository

Project: Launchly

5.2 Assessed File Listing

File	SHA-256
contracts/BondingCurve.sol	—
contracts/BondingCurveFactory.sol	—
contracts/MemeToken.sol	—
contracts/TokenFactory.sol	—
contracts/TokenFactoryWithCurve.sol	—

5.3 Technical Details

Detail	Value
Solidity Version	0.8.28
OpenZeppelin Version	4.9.6
Chainlink Contracts	^1.5.0
Framework	Hardhat 2.22.0
Target Chain	Binance Smart Chain (BSC) Mainnet / Testnet
Optimizer	Enabled (1000 runs, via IR, Paris EVM)

5.4 Out-of-Scope

Third-party dependencies and economic attacks

Front-end application and off-chain components

New features or implementations after the assessed commit

6. Assessment Summary & Findings Overview

All findings: 25 | Critical: 3 | High: 6 | Medium: 4 | Low: 11 | Informational: 1

ID	Security Analysis	Risk Level	Remediation Date
LOT-01	sell() ignores minQuoteOut slippage parameter	HIGH	Open
LOT-02	TokenFactoryWithCurve.createTokenWithBondingCurve() reverts for external callers	CRITICAL	Open
LOT-03	curveAllocation not scaled by token decimals	CRITICAL	Open
LOT-04	Chainlink oracle missing staleness checks	HIGH	Open
LOT-05	Bonding curve not excluded from MemeToken anti-sniper limits	HIGH	Open
LOT-06	Owner can drain all BNB and tokens via rescue functions	HIGH	Open
LOT-07	External self-call in createDefaultTokenWithCurve changes msg.sender context	HIGH	Open
LOT-08	Inconsistent term2 formula between buy and sell for sub-WAD amounts	LOW	Open
LOT-09	calculateCurveParams fetches BNB price but does not use it	MEDIUM	Open
LOT-10	ERC-20 approve() front-running race condition	INFORMATIONAL	Open
LOT-11	TokenFactory.createTokenWithBondingCurve does not fund the bonding curve	MEDIUM	Open
LOT-12	No bounds validation on P0 and m curve parameters	LOW	Open

ID	Security Analysis	Risk Level	Remediation Date
LOT-13	Unused event FeeRecipientChanged	LOW	Open
LOT-14	Unused state variable bondingCurveImplementation	LOW	Open
LOT-15	Unused constant TRADABLE_TOKENS	LOW	Open
LOT-16	Unnecessary nonReentrant modifier on token transfers	LOW	Open
LOT-17	P0 and m not declared as immutable	LOW	Open
LOT-18	Extra /WAD scaling nullifies quadratic term in pricing formula	CRITICAL	Open
LOT-19	Bonding curve math assumes 18-decimal tokens	HIGH	Open
LOT-20	Immutable feeRecipient can permanently disable buy and sell operations	MEDIUM	Open
LOT-21	buyWithBNB() lacks deadline parameter for transaction expiry	MEDIUM	Open
LOT-23	updateAntiSniperSettings event emits unscaled values	LOW	Open
LOT-24	State updates after external calls violate checks-effects-interactions pattern	LOW	Open
LOT-26	lastTransferTime updated unconditionally regardless of anti-sniper state	LOW	Open
LOT-27	Binary search in tokensForQuote may consume unpredictable gas	LOW	Open

7. Findings & Technical Details

7.1 sell() Ignores minQuoteOut Slippage Parameter

Severity: HIGH

Description

```
The sell() function in the BondingCurveBNB contract accepts a minQuoteOut
parameter intended to provide slippage protection for sellers. However, the
parameter is explicitly commented out and not enforced anywhere in the sell
logic. The netOut value is calculated but never validated against a minimum
threshold.
```

```
function sell(uint256 tokensIn, uint256 /* minQuoteOut */) external
nonReentrant {
```

```
The netOut is computed at line 468 but no require(netOut >= minQuoteOut, ...)
check exists. Sellers have zero slippage protection and are fully exposed to
sandwich attacks and MEV extraction.
```

Code Location

```
contracts/BondingCurve.sol : Line 457
```

Proof of Concept

```
A seller submits a sell() transaction with a reasonable minQuoteOut value
```

A MEV bot observes the pending transaction and front-runs it with a large sell order, reducing the price

The victim's sell executes at a significantly lower price than expected

The MEV bot back-runs with a buy order, profiting from the price differential

```
Since minQuoteOut is ignored, the victim's transaction succeeds regardless of
how unfavorable the received amount is
```

Impact

Sellers receive no protection against price manipulation

MEV bots can extract value from every sell transaction

Users may receive significantly less BNB than expected when selling tokens

BVSS

```
AO:A/AC:L/AX:L/C:N/I:H/A:N/D:H/Y:N/R:N/S:C (8.5)
```

Recommendation

```
It is recommended to uncomment the minQuoteOut parameter and add a slippage
check after fee deduction but before the BNB transfer. A deadline parameter
should also be introduced to prevent stale transactions from executing at
outdated prices, consistent with the pattern established by Uniswap V2/V3:
```

```
function sell(uint256 tokensIn, uint256 minQuoteOut, uint256 deadline)
external nonReentrant {
```

```
require(block.timestamp <= deadline, "expired");
```

```
require(tokensIn > 0, "zero tokens");
```

```
require(tokensIn <= sold, "cannot sell more than sold");
```

```
require(token.transferFrom(msg.sender, address(this), tokensIn), "
transferFrom failed");
```

```
uint256 grossOut = sellQuoteFor(sold, tokensIn);
```

```
require(grossOut > 0, "quote too small");
```

```
uint256 fee = (grossOut * FEE_BPS) / BPS_BASE;
```

```
uint256 netOut = grossOut - fee;
```

```
require(netOut >= minQuoteOut, "slippage");
```

```
require(address(this).balance >= grossOut, "insufficient liquidity");
```

```
// ... rest of payout logic ...
```

```
}
```

7.2 TokenFactoryWithCurve.createTokenWithBondingCurve() Reverts for External Callers

Severity: CRITICAL

Description

```
The createTokenWithBondingCurve() function in the TokenFactoryWithCurve
contract contains a critical logic error in its token distribution flow. At
line 57, the MemeToken is created with all tokens minted to msg.sender (the
external caller). Subsequently, at line 84, the function attempts to transfer
tokens from the factory contract itself, which holds a zero balance.
```

```
MemeToken token = new MemeToken(
```

```
_name, _symbol, _decimals,
```

```
msg.sender, // tokens minted to user
```

```
_totalSupply, ...
```

```
);
```

The factory then attempts:

```
require(token.transfer(curveAddress, curveAllocation), "transfer failed");
```

```
Inside MemeToken.transfer, msg.sender is the factory contract, and
balanceOf[factory] is zero. The require(balanceOf[msg.sender] >= value, ...)
check at MemeToken line 125 fails. This function always reverts and is
completely unusable.
```

Code Location

```
contracts/TokenFactoryWithCurve.sol : Lines 53–84
```

Impact

The primary factory function for creating tokens with bonding curves is entirely non-functional

No user can deploy a token-curve pair through this contract

```
The TokenFactoryWithCurve contract is effectively dead code on-chain
```

BVSS

```
AO:A/AC:L/AX:L/C:N/I:C/A:C/D:N/Y:N/R:F/S:C (9.0)
```

Recommendation

```
It is recommended to mint tokens to the factory contract, then distribute
atomically. SafeERC20 from OpenZeppelin should be used for the transfers:
```

```
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
```

```
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
```

```
using SafeERC20 for IERC20;
```

```
function createTokenWithBondingCurve(...) external returns (address
tokenAddress, address curveAddress) {
```

```
uint256 curveAllocation = ((_totalSupply * 80) / 100) * (10 ** _decimals);
```

```
uint256 rawTotalSupply = _totalSupply * (10 ** _decimals);
```

```
uint256 creatorAllocation = rawTotalSupply - curveAllocation;
```

```
// Mint to factory (address(this)), NOT msg.sender
```

```
MemeToken token = new MemeToken(
```

```
_name, _symbol, _decimals,
address(this),
_totalSupply, ...
);
tokenAddress = address(token);
BondingCurveBNB curve = new BondingCurveBNB(
tokenAddress, _P0_wad, _m_wad, curveAllocation, feeRecipient, msg.sender
);
curveAddress = address(curve);
// Distribute tokens atomically
IERC20(tokenAddress).safeTransfer(curveAddress, curveAllocation);
IERC20(tokenAddress).safeTransfer(msg.sender, creatorAllocation);
// Transfer token ownership to creator
token.transferOwnership(msg.sender);
// Exclude bonding curve from anti-sniper limits
```

```
token.setExcludedFromLimits(curveAddress, true);
```

```
emit TokenCreated(tokenAddress, msg.sender, _name, _symbol);
```

```
emit BondingCurveCreated(tokenAddress, curveAddress);
```

```
}
```

7.3 curveAllocation Not Scaled by Token Decimals

Severity: CRITICAL

Description

```
The TokenFactoryWithCurve contract calculates curveAllocation without scaling
by token decimals at line 49:
```

```
uint256 curveAllocation = (_totalSupply * 80) / 100; // 800M tokens
```

```
For a token with 18 decimals, _totalSupply is 1_000_000_000 and
curveAllocation is computed as 800_000_000 (8e8). However, the actual raw
total supply is 1_000_000_000 * 10^18 = 1e27, and the correct curveAllocation
should be 800_000_000 * 10^18 = 8e26.
```

```
This contrasts with BondingCurveFactory.sol line 203, which correctly scales:
```

```
uint256 curveAllocation = CURVE_TOKENS * (10 ** _tokenDecimals); // Correct
```

```
The bonding curve receives a curveAllocation that is 1e18 times too small.
The price formula P(s) = P0 + m * s / curveAllocation produces prices
amplified by approximately 1e18, rendering the curve immediately unusable.
```

Code Location

```
contracts/TokenFactoryWithCurve.sol : Line 49
```

Impact

The bonding curve pricing model produces wildly incorrect prices

Token purchases through the curve cost orders of magnitude more than intended

The curve becomes economically non-functional immediately upon deployment

BVSS

```
AO:A/AC:L/AX:L/C:N/I:C/A:H/D:H/Y:N/R:N/S:C (9.5)
```

Recommendation

```
It is recommended to scale curveAllocation by token decimals, consistent with
the approach used in BondingCurveFactory:
```

```
uint256 curveAllocation = ((_totalSupply * 80) / 100) * (10 ** _decimals);
```

7.4 Chainlink Oracle Missing Staleness Checks

Severity: HIGH

Description

```
The getLatestBNBPrice() function in BondingCurveFactory only validates that
rawPrice > 0. The roundId, updatedAt, and answeredInRound return values from
latestRoundData() are discarded without any staleness or round-completeness
validation:
```

```
(
```

```
/* uint80 roundID */,
```

```
int256 rawPrice,
```

```
/* uint startedAt */,
```

```
/* uint timeStamp */,
```

```
/* uint80 answeredInRound */
```

```
) = bnbPriceFeed.latestRoundData();
```

```
require(rawPrice > 0, "Invalid price");
```

Without staleness checks, the contract may operate on outdated price data during Chainlink outages or delayed oracle updates, leading to incorrect curve parameter calculations.

Code Location

```
contracts/BondingCurveFactory.sol : Lines 60–75
```

Impact

Curve parameters may be calculated using stale BNB/USD prices

During oracle outages, the contract continues operating with potentially hours-old data

Incorrect pricing can lead to mispriced bonding curves and user losses

BVSS

```
AO:A/AC:M/AX:M/C:N/I:H/A:N/D:M/Y:N/R:P/S:C (7.0)
```

Recommendation

```
It is recommended to add staleness validation using the updatedAt timestamp.
The BNB/USD feed on BSC has a heartbeat of 60 seconds. A staleness threshold
should be configured to provide adequate buffer. Note that the
answeredInRound >= roundID check pattern is deprecated by Chainlink; the
correct modern approach uses only the updatedAt staleness check:
```

```
uint256 public constant STALENESS_THRESHOLD = 3600; // 1 hour
```

```
function getLatestBNBPrice() public view returns (uint256 price) {
```

```
(
```

```
uint80 roundId,
```

```
int256 rawPrice,
```

```
/* uint startedAt */,
```

```
uint256 updatedAt,
```

```
/* uint80 answeredInRound */
```

```
) = bnbPriceFeed.latestRoundData();
```

```
require(rawPrice > 0, "Invalid price");
```

```
require(roundId != 0, "Invalid round");
```

```
require(updatedAt != 0 && updatedAt <= block.timestamp, "Invalid timestamp");
```

```
require(block.timestamp - updatedAt <= STALENESS_THRESHOLD, "Stale price");
```

```
price = uint256(rawPrice) * (10 ** 10);
```

```
}
```

7.5 Bonding Curve Not Excluded from MemeToken Anti-Sniper Limits

Severity: HIGH

Description

```
The MemeToken constructor at line 79 only excludes the token creator from
anti-sniper protections:
```

```
excludedFromLimits[_creator] = true;
```

```
When BondingCurveBNB.buyWithBNB() executes token.transfer(msg.sender, ds) at
line 442, inside MemeToken.transfer the msg.sender is the bonding curve
contract address. Since the bonding curve is not excluded from anti-sniper
limits:
```

```
lastTransferTime[bondingCurve] is updated on every buy (line 136)
```

```
The 30-second cooldownPeriod (line 131) applies globally to the bonding curve
address
```

Only one buy transaction can execute every 30 seconds across all users

```
For sell operations via transferFrom (line 462), the cooldown applies
per-seller, which is less impactful but still incorrectly constrains
legitimate users.
```

Code Location

```
contracts/MemeToken.sol : Lines 109, 128
```

```
contracts/BondingCurve.sol : Lines 442, 462
```

Impact

The bonding curve is globally rate-limited to one buy per 30-second window across all users

During high-demand periods, the vast majority of buy transactions will revert

The curve becomes practically unusable for its intended purpose of open market trading

BVSS

```
AO:A/AC:L/AX:L/C:N/I:M/A:H/D:N/Y:N/R:F/S:C (7.5)
```

Recommendation

```
It is recommended to exclude the bonding curve from anti-sniper limits during
the factory creation flow. Since setExcludedFromLimits is an onlyOwner
function, the factory must be the token owner at the time of the call:
```

```
// Factory mints to itself, so factory is owner
```

```
token.setExcludedFromLimits(address(curve), true);
```

```
// Then transfer ownership to creator
```

```
token.transferOwnership(msg.sender);
```

```
For TokenFactory.sol where the creator must manually fund the curve, the
creator should call token.setExcludedFromLimits(bondingCurveAddress, true)
after creation.
```

7.6 Owner Can Drain All BNB and Tokens via Rescue Functions

Severity: HIGH

Description

```
The BondingCurveBNB contract contains three unrestricted owner-only functions
that can drain all contract assets:
```

```
function withdrawToken(address to, uint256 amount) external onlyOwner { ... }
// line 383
```

```
function rescueERC20(address tokenAddr, address to, uint256 amount) external
onlyOwner { ... } // line 389
```

```
function rescueBNB(address payable to, uint256 amount) external onlyOwner { .
.. } // line 392
```

```
Additionally, rescueERC20 can be called with the curve's own token address,
bypassing any intended separation between withdrawToken and rescueERC20. The
markCurveFinished() function (line 399) can halt all buying unilaterally.
Together, these functions enable a complete rug-pull vector.
```

Code Location

```
contracts/BondingCurve.sol : Lines 383–396
```

Impact

The contract owner can extract all BNB liquidity backing the bonding curve

The contract owner can extract all deposited tokens

Users who have purchased tokens through the curve have no assurance that liquidity will remain available for sells

This constitutes a centralization risk and rug-pull vector

BVSS

```
AO:S/AC:L/AX:L/C:N/I:H/A:H/D:C/Y:N/R:N/S:C (8.0)
```

Recommendation

```
It is recommended to restrict rescue functions to prevent extraction of
active liquidity. The rescueERC20 function should be prevented from operating
on the curve's own token, and rescueBNB should enforce solvency constraints.
For stronger protection, consider wrapping owner operations behind an
OpenZeppelin TimelockController with a 24–48 hour delay:
```

```
function withdrawToken(address to, uint256 amount) external onlyOwner {
```

```
require(to != address(0), "zero");
```

```
uint256 activeBalance = curveAllocation - sold;
```

```
uint256 currentBalance = token.balanceOf(address(this));
```

```
require(currentBalance - amount >= activeBalance, "cannot withdraw active
tokens");
```

```
IERC20(address(token)).safeTransfer(to, amount);
```

```
}
```

```
function rescueERC20(address tokenAddr, address to, uint256 amount) external
onlyOwner {
```

```
require(tokenAddr != address(token), "cannot rescue curve token");
```

```
IERC20(tokenAddr).safeTransfer(to, amount);
```

```
}
```

```
function rescueBNB(address payable to, uint256 amount) external onlyOwner {
```

```
require(to != payable(address(0)), "zero");
```

```
uint256 requiredBNB = sellQuoteFor(sold, sold);
```

```
require(address(this).balance - amount >= requiredBNB, "would make curve
insolvent");
```

```
(bool sent,) = to.call{value: amount}("");
```

```
require(sent, "BNB send failed");
```

```
}
```

7.7 External Self-Call in createDefaultTokenWithCurve Changes msg.sender Context

Severity: HIGH

Description

```
The createDefaultTokenWithCurve() function in TokenFactoryWithCurve uses an
external self-call pattern at line 100:
```

```
function createDefaultTokenWithCurve(...) external returns (...) {
```

```
return this.createTokenWithBondingCurve(...); // external self-call
```

```
}
```

```
The this.createTokenWithBondingCurve(...) syntax performs an external call to
the same contract. Inside createTokenWithBondingCurve, msg.sender becomes the
factory contract itself rather than the original user. This results in:
```

Tokens being minted to the factory contract

The bonding curve owner being set to the factory contract

The original user receiving nothing

Code Location

```
contracts/TokenFactoryWithCurve.sol : Line 100
```

Impact

```
The convenience function createDefaultTokenWithCurve does not function as
intended
```

Users calling this function lose ownership of both the token and the bonding curve

Token and curve ownership becomes permanently locked in the factory contract

BVSS

```
AO:A/AC:L/AX:L/C:N/I:H/A:H/D:H/Y:N/R:N/S:C (8.5)
```

Recommendation

```
It is recommended to refactor the shared logic into an internal function to
preserve msg.sender context:
```

```
function createTokenWithBondingCurve(
```

```
string memory _name, string memory _symbol, uint8 _decimals,
```

```
uint256 _totalSupply, uint256 _P0_wad, uint256 _m_wad, uint256 /*
_bnbPriceUSD */
```

```
) external returns (address tokenAddress, address curveAddress) {
```

```
return _createTokenWithBondingCurve(
```

```
_name, _symbol, _decimals, _totalSupply, _P0_wad, _m_wad
);
}
function createDefaultTokenWithCurve(
string memory _name, string memory _symbol,
uint256 _P0_wad, uint256 _m_wad, uint256 /* _bnbPriceUSD */
) external returns (address tokenAddress, address curveAddress) {
return _createTokenWithBondingCurve(
_name, _symbol, 18, 1_000_000_000, _P0_wad, _m_wad
);
}
function _createTokenWithBondingCurve(
// ... all params ...
) internal returns (address tokenAddress, address curveAddress) {
// All logic here — msg.sender is preserved correctly
```

```
}
```

7.8 Inconsistent term2 Formula Between Buy and Sell for Sub-WAD Amounts

Severity: LOW

Description

```
In the ds < WAD (sub-token) branches of the bonding curve pricing functions,
the term2 quadratic calculation is inconsistent between the buy and sell
paths.
```

```
buyQuoteFor (line 117) — correct formula:
```

```
term2 = (mOverWad * dsSquaredOverCurveAlloc) / 2;
```

```
sellQuoteFor (line 188) — incorrect extra /WAD:
```

```
term2 = (mOverWad * dsSquaredOverCurveAlloc) / (2 * WAD);
```

```
The sell path term2 is 1e18 times smaller than the buy path for identical
inputs. Since sellReturn = term1 - term2, a smaller term2 means sellers
receive more than they should. Practical impact is limited because for ds <
1e18, integer division typically rounds term2 to zero in both paths, but the
code correctness violation is real.
```

Code Location

```
contracts/BondingCurve.sol : Line 117 vs Line 188
```

BVSS

```
AO:A/AC:H/AX:H/C:N/I:L/A:N/D:L/Y:N/R:F/S:U (2.5)
```

Recommendation

```
It is recommended to fix the sell path to match the buy path formula. A fuzz
test invariant verifying buyQuoteFor(s, ds) >= sellQuoteFor(s + ds, ds) for
```

```
all valid input ranges should also be added:
```

```
// sellQuoteFor, ds < WAD branch:
```

```
if (dsOverWad == 0) {
```

```
uint256 mOverWad = m / WAD;
```

```
uint256 dsSquaredOverCurveAlloc = (ds * ds) / curveAllocation;
```

```
term2 = (mOverWad * dsSquaredOverCurveAlloc) / 2;
```

```
}
```

7.9 calculateCurveParams Fetches BNB Price but Does Not Use It

Severity: MEDIUM

Description

```
The calculateCurveParams() function in BondingCurveFactory fetches
bnbPriceUSD via Chainlink at line 91 but only returns it without using it in
any computation. The curve parameters p0_wad and m_wad are derived from
hardcoded constants:
```

```
p0_wad = (5798 * 1e18) / 1e12 = 5798000000 (line 101)
```

```
m_wad is derived from the hardcoded target of 0.99 BNB for 147,334,426 tokens
(lines 111–112)
```

The stated goal of targeting $5,000–$76,963 USD market cap is not enforced. Curve parameters remain identical regardless of BNB price fluctuations.

Code Location

```
contracts/BondingCurveFactory.sol : Lines 91–182
```

BVSS

```
AO:A/AC:L/AX:M/C:N/I:M/A:N/D:L/Y:N/R:P/S:U (5.0)
```

Recommendation

```
It is recommended to use the fetched BNB price dynamically to calculate P0
and m to achieve target USD market caps at the current BNB price. If the
protocol intends fixed BNB-denominated pricing, the Chainlink oracle
dependency should be removed entirely:
```

```
function calculateCurveParams() public view returns (
```

```
uint256 p0_wad, uint256 m_wad, uint256 bnbPriceUSD
```

```
) {
```

```
bnbPriceUSD = getLatestBNBPrice();
```

```
require(bnbPriceUSD > 0, "Invalid BNB price");
```

```
uint256 totalTokens = CURVE_TOKENS * WAD;
```

```
p0_wad = (START_MARKET_CAP_USD * WAD * WAD) / (totalTokens * bnbPriceUSD /
WAD);
```

```
uint256 pEnd_wad = (END_MARKET_CAP_USD * WAD * WAD) / (totalTokens *
bnbPriceUSD / WAD);
```

```
m_wad = pEnd_wad - p0_wad;
```

```
}
```

7.10 ERC-20 approve() Front-Running Race Condition

Severity: INFORMATIONAL

Description

```
The approve() function in MemeToken follows the standard ERC-20 set-and-emit
pattern:
```

```
function approve(address spender, uint256 value) external returns (bool) {
```

```
allowance[msg.sender][spender] = value;
```

```
emit Approval(msg.sender, spender, value);
```

```
return true;
```

```
}
```

This is the well-known approve race condition where a spender can front-run an allowance change to spend both the old and new allowance. However, this is the correct and standard ERC-20 implementation per EIP-20.

Code Location

```
contracts/MemeToken.sol : Lines 91–95
```

BVSS

```
AO:A/AC:H/AX:H/C:N/I:L/A:N/D:N/Y:N/R:F/S:U (1.5)
```

Recommendation

```
The current approve() implementation matches the EIP-20 specification. This
is a known, well-documented ERC-20 design characteristic. OpenZeppelin
removed increaseAllowance() and decreaseAllowance() from the ERC20 base
contract in v5.0.0 due to the additional phishing attack surface these
methods created. Consider implementing EIP-2612 permit() for gasless,
signature-based approvals to eliminate the approve race for supported wallets.
```

7.11 TokenFactory.createTokenWithBondingCurve Does Not Fund the Bonding Curve

Severity: MEDIUM

Description

```
The createTokenWithBondingCurve function in TokenFactory creates a token and
bonding curve but leaves funding as a manual multi-step process:
```

```
// Note: Token creator owns all tokens initially (1B tokens)
```

```
// Creator needs to manually deposit 800M tokens to the bonding curve after
creation
```

This requires two separate manual transactions:

```
token.approve(bondingCurveAddress, 800_000_000 * 10^decimals)
```

```
bondingCurve.depositCurveTokens(800_000_000 * 10^decimals)
```

```
If the creator omits these steps, the bonding curve exists on-chain but has
no tokens. Any buyWithBNB() call will fail, and the curve appears broken to
users.
```

Code Location

```
contracts/TokenFactory.sol : Lines 117–172
```

BVSS

```
AO:S/AC:L/AX:M/C:N/I:M/A:M/D:N/Y:N/R:F/S:U (4.5)
```

Recommendation

It is recommended to automate funding in the factory by applying the mint-to-factory pattern: mint tokens to the factory, transfer the curve allocation atomically, then transfer the remainder and ownership to the creator. If manual funding is the intended flow, a guard should be added to prevent buying from an unfunded curve:

```
function buyWithBNB(uint256 minTokensOut) external payable nonReentrant {
```

```
require(!curveFinished, "curve finished");
```

```
require(token.balanceOf(address(this)) > 0, "curve not funded");
```

```
// ...
```

```
}
```

7.12 No Bounds Validation on P0 and m Curve Parameters

Severity: LOW

Description

```
The TokenFactoryWithCurve contract accepts _P0_wad and _m_wad parameters
directly from callers with no bounds checking:
```

```
uint256 _P0_wad, // Initial price in BNB wei (WAD scaled)
```

```
uint256 _m_wad, // Slope in BNB wei per token (WAD scaled)
```

```
A creator can set P0 = 0 (free tokens), m = 0 (flat curve with no price
progression), or extreme values that may cause arithmetic overflow during
quote calculations.
```

Code Location

```
contracts/TokenFactoryWithCurve.sol : Lines 39–40
```

BVSS

```
AO:S/AC:L/AX:L/C:N/I:L/A:L/D:N/Y:N/R:F/S:U (3.0)
```

Recommendation

It is recommended to add reasonable bounds validation:

```
uint256 public constant MIN_P0 = 1;
```

```
uint256 public constant MAX_P0 = 1e18;
```

```
uint256 public constant MIN_M = 1;
```

```
uint256 public constant MAX_M = 100e18;
```

```
function createTokenWithBondingCurve(...) external returns (...) {
```

```
require(_P0_wad >= MIN_P0 && _P0_wad <= MAX_P0, "P0 out of range");
```

```
require(_m_wad >= MIN_M && _m_wad <= MAX_M, "m out of range");
```

```
// ...
```

```
}
```

7.13 Unused Event FeeRecipientChanged

Severity: LOW

Description

```
The FeeRecipientChanged event is declared at line 42 of BondingCurveBNB but
is never emitted anywhere in the contract. The feeRecipient variable is
declared immutable (line 25) and set only in the constructor, making a change
event semantically impossible.
```

```
event FeeRecipientChanged(address indexed newRecipient);
```

Code Location

```
contracts/BondingCurve.sol : Line 42
```

BVSS

```
AO:A/AC:L/AX:L/C:N/I:N/A:N/D:N/Y:N/R:F/S:U (2.0)
```

Recommendation

It is recommended to remove the unused event declaration:

```
- event FeeRecipientChanged(address indexed newRecipient);
```

7.14 Unused State Variable bondingCurveImplementation

Severity: LOW

Description

```
The bondingCurveImplementation state variable is declared at line 10 of
TokenFactoryWithCurve and has a setter function
setBondingCurveImplementation() at line 21, but is never read in any
function. The factory creates curves via new BondingCurveBNB(...) directly at
line 71.
```

Code Location

```
contracts/TokenFactoryWithCurve.sol : Lines 10, 21
```

BVSS

```
AO:A/AC:L/AX:L/C:N/I:N/A:N/D:N/Y:N/R:F/S:U (2.0)
```

Recommendation

```
It is recommended to remove the variable and its setter to save a storage
slot and reduce the contract attack surface. If an upgrade pattern is
intended for the future, a proper proxy pattern such as OpenZeppelin Clones
should be used:
```

```
- address public bondingCurveImplementation;
```

```
- function setBondingCurveImplementation(address _implementation) external
onlyOwner {
```

```
- bondingCurveImplementation = _implementation;
```

```
- }
```

7.15 Unused Constant TRADABLE_TOKENS

Severity: LOW

Description

```
The TRADABLE_TOKENS constant is declared at line 21 of BondingCurveFactory
but is never referenced in any function:
```

```
uint256 public constant TRADABLE_TOKENS = 800_000_000;
```

```
The factory uses CURVE_TOKENS (1,000,000,000) for curve allocation at line
203.
```

Code Location

```
contracts/BondingCurveFactory.sol : Line 21
```

BVSS

```
AO:A/AC:L/AX:L/C:N/I:N/A:N/D:N/Y:N/R:F/S:U (2.0)
```

Recommendation

It is recommended to remove the unused constant:

```
- uint256 public constant TRADABLE_TOKENS = 800_000_000;
```

7.16 Unnecessary nonReentrant Modifier on Token Transfers

Severity: LOW

Description

```
Both transfer() and transferFrom() in MemeToken use the nonReentrant modifier:
```

```
function transferFrom(address from, address to, uint256 value) external
nonReentrant returns (bool) { ... }
```

```
function transfer(address to, uint256 value) external nonReentrant returns
(bool) { ... }
```

These functions only perform balance checks, balance updates, allowance updates, anti-sniper checks (pure storage reads/writes), and event emissions. No external calls are made. There is no reentrancy vector, and the modifier adds approximately 2,600 gas overhead (cold SSTORE for the reentrancy lock) to every token transfer.

```
Additionally, nonReentrant prevents transfer and transferFrom from being
called within any other nonReentrant function on the same contract in the
same call stack, which is an unnecessary restriction.
```

Code Location

```
contracts/MemeToken.sol : Lines 98, 124
```

BVSS

```
AO:A/AC:L/AX:L/C:N/I:N/A:N/D:N/Y:L/R:F/S:U (2.0)
```

Recommendation

```
It is recommended to remove nonReentrant from both transfer functions:
```

```
- function transfer(address to, uint256 value) external nonReentrant returns
(bool) {
```

```
+ function transfer(address to, uint256 value) external returns (bool) {
```

```
- function transferFrom(address from, address to, uint256 value) external
nonReentrant returns (bool) {
```

```
+ function transferFrom(address from, address to, uint256 value) external
returns (bool) {
```

7.17 P0 and m Not Declared as immutable

Severity: LOW

Description

```
The bonding curve parameters P0 and m are declared as regular storage
variables but are only set in the constructor and never modified:
```

```
uint256 public P0; // line 32
```

```
uint256 public m; // line 33
```

```
Every call to priceAt(), buyQuoteFor(), sellQuoteFor(), and tokensForQuote()
reads these values from storage via SLOAD (~2,100 gas cold / ~100 gas warm).
Using the immutable keyword embeds the values in contract bytecode, making
reads effectively free.
```

Code Location

```
contracts/BondingCurve.sol : Lines 32–33
```

BVSS

```
AO:A/AC:L/AX:L/C:N/I:N/A:N/D:N/Y:L/R:F/S:U (2.0)
```

Recommendation

```
It is recommended to declare both variables as immutable:
```

```
- uint256 public P0;
```

```
- uint256 public m;
```

```
+ uint256 public immutable P0;
```

```
+ uint256 public immutable m;
```

```
The gas savings compound significantly for active curves, as tokensForQuote
calls buyQuoteFor repeatedly in a binary search loop.
```

7.18 Extra /WAD Scaling Nullifies Quadratic Term in Pricing Formula

Severity: CRITICAL

Description

```
In the ds >= WAD branches of both buyQuoteFor() and sellQuoteFor(), the
quadratic term (term2) has an extraneous division by WAD that reduces it by a
factor of 1e18.
```

```
At line 132 (buyQuoteFor, primary path):
```

```
term2 = (temp2 / curveAllocationOverWad) / (2 * WAD);
```

```
Where temp2 = m * dsOverWad * dsOverWad.
```

The correct mathematical derivation yields:

```
term2 = m * dsOverWad^2 / (2 * curveAllocationOverWad)
```

Therefore the correct code should be:

```
term2 = (temp2 / curveAllocationOverWad) / 2;
```

```
The extra / WAD makes term2 approximately 1e18 times too small. The bonding
curve effectively becomes a flat-price curve at P0, with the quadratic price
progression entirely nullified. This affects all eight code branches across
both functions at lines 132, 135, 147, 159, 200, 203, 215, and 227.
```

Code Location

```
contracts/BondingCurve.sol : Lines 132, 135, 147, 159, 200, 203, 215, 227
```

Proof of Concept

```
The bonding curve is deployed with P0 = 5798000000 and m = 50000000000
```

```
A user buys tokens; term2 is divided by an extra 1e18, making the price
increase negligible
```

```
The user receives tokens at approximately the flat price P0
```

```
The user immediately sells the tokens; sellQuoteFor has the same bug,
producing a near-identical return value
```

A buy-sell round-trip at any supply level costs approximately the same, enabling arbitrage extraction of BNB from the pool

Impact

The bonding curve does not function as a bonding curve — it produces flat pricing

Buy-sell round-trips can extract BNB from the pool through arbitrage

The entire economic model of price discovery through bonding is broken

All protocol revenue from price progression is lost

BVSS

```
AO:A/AC:L/AX:L/C:N/I:C/A:N/D:C/Y:N/R:N/S:C (10.0)
```

Recommendation

```
It is recommended to remove the extra / WAD in all affected branches. Given
the complexity and number of branches, the quadratic calculation should be
extracted into a shared helper function:
```

```
/// @dev Calculate quadratic term: m * ds^2 / (2 * curveAllocation * WAD)
```

```
/// @notice Correct formula derivation:
```

```
/// term2 = m * ds^2 / (2 * curveAllocation * WAD)
```

```
/// With ds = dsOverWad * WAD, curveAllocation = curveAllocationOverWad * WAD:
```

```
/// term2 = m * dsOverWad^2 * WAD^2 / (2 * curveAllocationOverWad * WAD * WAD)
```

```
/// = m * dsOverWad^2 / (2 * curveAllocationOverWad)
```

```
function _quadraticTerm(uint256 ds) internal view returns (uint256) {
if (ds == 0 || curveAllocation == 0) return 0;
uint256 dsOverWad = ds / WAD;
uint256 curveAllocationOverWad = curveAllocation / WAD;
if (dsOverWad == 0) {
// Sub-token amounts: direct calculation
uint256 mOverWad = m / WAD;
uint256 dsSquaredOverCurveAlloc = (ds * ds) / curveAllocation;
return (mOverWad * dsSquaredOverCurveAlloc) / 2;
}
if (curveAllocationOverWad == 0) return 0;
// Primary path: term2 = m * dsOverWad^2 / (2 * curveAllocationOverWad)
if (m <= type(uint256).max / dsOverWad) {
uint256 temp = m * dsOverWad;
if (temp <= type(uint256).max / dsOverWad) {
uint256 temp2 = temp * dsOverWad;
```

```
return (temp2 / curveAllocationOverWad) / 2; // NO extra / WAD
}
}
// Fallback: divide m by WAD first, then multiply by WAD to restore scaling
// mOverWad = m / WAD loses one factor, so we must compensate
// term2 = m * dsOverWad^2 / (2 * curveAllocationOverWad)
// = (m / WAD) * dsOverWad^2 * WAD / (2 * curveAllocationOverWad)
uint256 mOverWad = m / WAD;
uint256 numerator = mOverWad * dsOverWad * dsOverWad;
// Multiply by WAD before dividing to restore the lost WAD factor
return (numerator * WAD) / (2 * curveAllocationOverWad);
}
```

Invariant tests should be added to verify no arbitrage is possible:

```
function testNoArbitrage(uint256 s, uint256 ds) public {
vm.assume(ds > 0 && ds <= curveAllocation - s);
uint256 buyCost = curve.buyQuoteFor(s, ds);
uint256 sellReturn = curve.sellQuoteFor(s + ds, ds);
```

```
assertGe(buyCost, sellReturn, "arbitrage possible");
```

```
}
```

7.19 Bonding Curve Math Assumes 18-Decimal Tokens

Severity: HIGH

Description

```
The BondingCurveBNB contract hardcodes all pricing arithmetic around WAD =
1e18, implicitly assuming token raw units have 18 decimals. However,
TokenFactory.createTokenWithBondingCurve() allows arbitrary decimals (line
131: require(_decimals > 0, ...)).
```

```
Key references in BondingCurveBNB:
```

```
// buyQuoteFor line 100:
```

```
uint256 term1 = (b * ds) / WAD;
```

```
// buyQuoteFor line 106:
```

```
uint256 dsOverWad = ds / WAD;
```

```
// sellQuoteFor line 175:
```

```
uint256 term1 = (b * ds) / WAD;
```

For a 6-decimal token:

```
1 full token = 1e6 raw units
```

```
dsOverWad = 1e6 / 1e18 = 0 (rounds to zero)
```

```
term1 = (b * 1e6) / 1e18 produces negligibly small values
```

The curve becomes economically non-functional for any token with decimals other than 18.

Code Location

```
contracts/TokenFactory.sol : Line 131
```

```
contracts/BondingCurveFactory.sol : Line 203
```

```
contracts/BondingCurve.sol : Lines 100, 175
```

BVSS

```
AO:A/AC:L/AX:L/C:N/I:H/A:H/D:M/Y:N/R:N/S:C (8.0)
```

Recommendation

It is recommended to enforce 18 decimals in all factory paths that create bonding curves. This is the simplest and safest approach, as the vast majority of BSC tokens use 18 decimals. Token creation without bonding curves can continue to support arbitrary decimals:

```
// In TokenFactory.createTokenWithBondingCurve:
```

```
require(_decimals == 18, "bonding curves require 18 decimals");
```

```
// In BondingCurveFactory.createBondingCurve:
```

```
require(_tokenDecimals == 18, "bonding curves require 18 decimals");
```

```
// In TokenFactoryWithCurve.createTokenWithBondingCurve:
```

```
require(_decimals == 18, "bonding curves require 18 decimals");
```

7.20 Immutable feeRecipient Can Permanently Disable Buy and Sell Operations

Severity: MEDIUM

Description

```
The feeRecipient address in BondingCurveBNB is declared as immutable at line
25 and can never be changed after deployment. The constructor at line 53 only
validates that the address is non-zero but does not verify that it can
receive BNB.
```

```
Both buyWithBNB() and sell() contain a mandatory fee transfer to feeRecipient
that uses a require on the transfer result:
```

```
if (fee > 0) {
```

```
(bool ok,) = payable(feeRecipient).call{value: fee}("");
```

```
require(ok, "fee transfer failed");
```

```
}
```

```
If feeRecipient is set to a contract address that reverts on BNB receipt (a
contract without a receive() or fallback() function, or one that
intentionally reverts), every buy and sell transaction will permanently fail.
Since the variable is immutable, there is no administrative mechanism to
update it, and no recovery path exists. The bonding curve and all deposited
tokens become permanently locked.
```

Code Location

```
contracts/BondingCurve.sol : Lines 25, 53, 413–415, 478–480
```

Impact

```
All buy operations via buyWithBNB() permanently revert
```

```
All sell operations via sell() permanently revert
```

Tokens deposited in the bonding curve become irrecoverable through normal operations

BNB accumulated from prior purchases becomes inaccessible to sellers

```
The only recovery path is the owner calling rescueBNB() and withdrawToken()
to manually extract assets
```

BVSS

```
AO:S/AC:L/AX:L/C:N/I:N/A:C/D:H/Y:N/R:N/S:C (6.5)
```

Recommendation

```
It is recommended to add a validation check in the constructor to ensure
feeRecipient can receive BNB. Consider removing the immutable modifier and
adding a restricted setter function:
```

```
constructor(
```

```
address _token,
```

```
uint256 _P0_wad,
```

```
uint256 _m_wad,
```

```
uint256 _curveAllocation,
```

```
address _feeRecipient,
```

```
address _owner
```

```
) {
```

```
require(_token != address(0), "token 0");
```

```
require(_feeRecipient != address(0), "feeRecipient 0");
```

```
// Verify feeRecipient can receive BNB
```

```
(bool testSend,) = payable(_feeRecipient).call{value: 0}("");
```

```
require(testSend, "feeRecipient cannot receive BNB");
```

```
token = IERC20(_token);
```

```
P0 = _P0_wad;
```

```
m = _m_wad;
```

```
curveAllocation = _curveAllocation;
```

```
feeRecipient = _feeRecipient;
```

```
_transferOwnership(_owner);
```

```
}
```

The fee transfer could also be wrapped in a try-catch pattern that redirects fees to the contract itself if the primary recipient reverts, ensuring core buy/sell functionality is never blocked by a failing fee transfer.

7.21 buyWithBNB() Lacks deadline Parameter for Transaction Expiry

Severity: MEDIUM

Description

```
The buyWithBNB() function at line 408 accepts a minTokensOut parameter for
slippage protection but does not include a deadline parameter for transaction
expiry:
```

```
function buyWithBNB(uint256 minTokensOut) external payable nonReentrant {
```

```
Without a deadline, a buy transaction that remains pending in the mempool can
be executed at an arbitrarily later time. If the bonding curve price
increases significantly between submission and execution, the buyer may
```

```
receive fewer tokens than intended at the time of signing. While
minTokensOutprovides some protection against unfavorable prices, it does not
prevent astale transaction from executing at a price that the user would no
longerconsider acceptable.
```

This is the same class of vulnerability that contributed to the Four.Meme sandwich attack in March 2025, where stale transactions were front-run during high-volatility periods on BSC.

Code Location

```
contracts/BondingCurve.sol : Line 408
```

Impact

Pending buy transactions can be executed minutes, hours, or days after submission

Users may receive fewer tokens than they expected at the time of signing the transaction

MEV bots can selectively include stale transactions when doing so is profitable for sandwich extraction

On BSC where gas costs are low, even small-value stale transactions are worth exploiting

BVSS

```
AO:A/AC:L/AX:M/C:N/I:M/A:N/D:M/Y:N/R:P/S:U (5.0)
```

Recommendation

```
It is recommended to add a deadline parameter to buyWithBNB() consistent with
the standard established by Uniswap V2/V3 and modern DEX protocols:
```

```
function buyWithBNB(uint256 minTokensOut, uint256 deadline) external payable
nonReentrant {
```

```
require(block.timestamp <= deadline, "transaction expired");
```

```
require(!curveFinished, "curve finished");
```

```
require(msg.value > 0, "send BNB");
```

```
// ... rest of buy logic ...
```

```
}
```

```
The same pattern should also be applied to sell() for consistent transaction
expiry protection.
```

7.23 updateAntiSniperSettings Event Emits Unscaled Values

Severity: LOW

Description

```
The updateAntiSniperSettings() function in MemeToken stores maxWallet and
maxTransaction as scaled values (multiplied by 10 ** decimals) at lines
155–156, but the AntiSniperSettingsUpdated event at line 159 emits the
original unscaled input values:
```

```
maxWallet = _maxWallet * (10 ** decimals); // line 155: stores 1000000e18
```

```
maxTransaction = _maxTransaction * (10 ** decimals); // line 156: stores
500000e18
```

```
emit AntiSniperSettingsUpdated(_antiSniperEnabled, _maxWallet,
_maxTransaction, _cooldownPeriod);
```

```
// line 159: emits 1000000 (unscaled) ^^^^^^^^^^^ ^^^^^^^^^^^^^^^^
```

```
Off-chain consumers (block explorers, subgraphs, and frontend applications)
reading the AntiSniperSettingsUpdated event will observe values that are
10^decimals smaller than the actual enforced limits. This discrepancy can
cause incorrect UI displays and misleading information to users.
```

Code Location

```
contracts/MemeToken.sol : Lines 155–156, 159
```

BVSS

```
AO:A/AC:L/AX:L/C:N/I:L/A:N/D:N/Y:N/R:F/S:U (2.0)
```

Recommendation

It is recommended to emit the scaled (stored) values in the event to match what is actually enforced on-chain:

```
maxWallet = _maxWallet * (10 ** decimals);
```

```
maxTransaction = _maxTransaction * (10 ** decimals);
```

```
cooldownPeriod = _cooldownPeriod;
```

```
emit AntiSniperSettingsUpdated(
```

```
_antiSniperEnabled,
```

```
maxWallet, // emit stored (scaled) value
```

```
maxTransaction, // emit stored (scaled) value
```

```
_cooldownPeriod
```

```
);
```

7.24 State Updates After External Calls Violate Checks-Effects-Interactions Pattern

Severity: LOW

Description

```
Both buyWithBNB() and sell() in BondingCurveBNB update the sold state
variable after performing external calls, violating the
checks-effects-interactions (CEI) pattern.
```

```
In buyWithBNB(), the token transfer at line 442 precedes the state update at
line 443:
```

```
require(token.transfer(msg.sender, ds), "token transfer failed"); // line
442: external call
```

```
sold += ds; // line 443: state update
```

```
In sell(), multiple external BNB transfers at lines 474 and 479 precede the
state update at line 483:
```

```
(bool sent,) = payable(msg.sender).call{value: netOut}(""); // line 474:
external call
```

```
require(sent, "pay seller failed");
```

```
if (fee > 0) {
```

```
(bool ok,) = payable(feeRecipient).call{value: fee}(""); // line 479:
external call
```

```
require(ok, "fee transfer failed");
```

```
}
```

```
sold -= tokensIn; // line 483: state update
```

```
Both functions are protected by the nonReentrant modifier, and
MemeToken.transfer() does not make external calls in the current
implementation. However, the CEI violation represents a defense-in-depth
concern. If the token contract is ever upgraded to include transfer callbacks
(e.g., ERC-777 hooks) or if a different token is used with the bonding curve,
the reentrancy guard becomes the sole line of defense.
```

Code Location

```
contracts/BondingCurve.sol : Lines 442–443, 474–483
```

BVSS

```
AO:A/AC:H/AX:H/C:N/I:L/A:N/D:L/Y:N/R:F/S:U (2.0)
```

Recommendation

```
It is recommended to move the sold state update before external calls in both
functions:
```

```
// In buyWithBNB():
```

```
sold += ds; // state update FIRST
```

```
require(token.transfer(msg.sender, ds), "token transfer failed"); // external
call AFTER
```

```
// In sell():
```

```
sold -= tokensIn; // state update FIRST
```

```
(bool sent,) = payable(msg.sender).call{value: netOut}(""); // external calls
AFTER
```

```
require(sent, "pay seller failed");
```

7.26 lastTransferTime Updated Unconditionally Regardless of Anti-Sniper State

Severity: LOW

Description

```
In both transfer() and transferFrom() of MemeToken, the lastTransferTime
mapping is updated on every transfer regardless of whether anti-sniper
protection is enabled or whether the sender is excluded from limits.
```

```
In transfer() at line 136:
```

```
if (antiSniperEnabled && !excludedFromLimits[msg.sender]) {
```

```
// anti-sniper checks ...
}
balanceOf[msg.sender] -= value;
balanceOf[to] += value;
lastTransferTime[msg.sender] = block.timestamp; // line 136: ALWAYS updated
```

```
The same pattern exists in transferFrom() at line 117, where
lastTransferTime[from] is always updated.
```

```
This has two consequences. First, approximately 5,000 gas is wasted on the
SSTORE operation for every transfer when antiSniperEnabled is false, as the
timestamp value is never checked. Second, if anti-sniper is disabled and
later re-enabled, stale lastTransferTime values from the disabled period
persist. Users who transferred during the disabled period may be unexpectedly
blocked by the cooldown when protection is re-enabled.
```

Code Location

```
contracts/MemeToken.sol : Lines 117, 136
```

BVSS

```
AO:A/AC:L/AX:L/C:N/I:L/A:N/D:N/Y:N/R:F/S:U (2.0)
```

Recommendation

```
It is recommended to move the lastTransferTime update inside the anti-sniper
conditional block:
```

```
if (antiSniperEnabled && !excludedFromLimits[msg.sender]) {
```

```
require(value <= maxTransaction, "Transfer > maxTransaction");
```

```
require(balanceOf[to] + value <= maxWallet, "Recipient > maxWallet");
```

```
require(block.timestamp - lastTransferTime[msg.sender] >= cooldownPeriod, "
Cooldown active");
```

```
lastTransferTime[msg.sender] = block.timestamp; // only update when actively
enforcing
```

```
}
```

```
balanceOf[msg.sender] -= value;
```

```
balanceOf[to] += value;
```

7.27 Binary Search in tokensForQuote May Consume Unpredictable Gas

Severity: LOW

Description

```
The tokensForQuote() function at lines 246–369 contains three sequential
search phases, each calling buyQuoteFor() which performs complex WAD-scaled
arithmetic:
```

A primary binary search with a maximum of 256 iterations (line 279)

A power-of-two escalation search (lines 336–345) with up to approximately 88 iterations for the full uint256 range

A secondary binary search with a maximum of 100 iterations (line 352)

```
In the worst case, all three phases execute sequentially, resulting in up to
444 iterations of buyQuoteFor(). Each iteration involves multiple branches of
overflow-protected multiplication and division. The gas cost of a single
buyWithBNB() call is therefore highly variable and depends on the
relationship between the linear price estimate and the actual bonding curve
cost.
```

Code Location

```
contracts/BondingCurve.sol : Lines 246–369
```

BVSS

```
AO:A/AC:L/AX:L/C:N/I:N/A:L/D:N/Y:N/R:F/S:U (2.0)
```

Recommendation

It is recommended to add a gas-efficient early termination condition and reduce the maximum iteration counts. For a binary search over a 256-bit range, 128 iterations are sufficient to converge to a single value. A view function that returns an estimated gas cost for a given buy amount should be considered to allow frontends to warn users about high-gas transactions:

```
uint256 maxIterations = 128; // sufficient for full uint256 range
```